

Functional Programming

Project: Hoodle (part 1)

Assistant: Noah Van Es

`noah.van.es@vub.be`

The final grade of the course is determined by an oral exam and a project (each counts for 50% of the total grade for the course). The project consists out of two parts. A non-empty solution for both parts has to be submitted in order to get a grade. This document describes the first part of the project.

Project Description

The goal of this first programming assignment is to create a terminal-based application for scheduling meetings (similar to the popular Doodle¹ meeting application).

A “hoodle” is used to agree on a timeslot for an event with many participants. Specifically, the organiser of the event can create a hoodle, providing all possible timeslots for an event. Upon creating such a hoodle, the organiser receives an identifier which can be shared with other participants. These participants can then use this identifier to retrieve the hoodle and indicate for which timeslots they are available. Once all participants have filled in the hoodle, the idea is that the organiser can select the best timeslot for the event (usually, the one that satisfies all or most participants).

Practical Information

Submission. The deadline for this assignment is **June 1, 2026 (23:59)**. You submit all your code and other materials in a single ZIP-file through the Canvas platform. Optionally, you may include a **README.md** file in your archive to clarify certain aspects of your implementation that warrant further explanation (e.g., the structure of your project).

Grading. Note that your project will not only be graded on functionality, but also on the quality of your implementation (programming style, performance considerations, ...). In particular, **you will be graded on the correct use of FP concepts and abstractions discussed during the lectures and exercise sessions.**

¹<https://doodle.com>

Individual Work. This project is to be made strictly individually and on your own. This means that you must create your project on an independent basis and you must be able to explain your work, reimplement it under supervision, and defend your solution. Copying work (code, text, etc.) from or sharing it with third parties (e.g., fellow students, generative AI tools, websites, GitHub, etc.) is not allowed. Electronic tools are used to compare all submissions with online resources and with each other, even across academic years. Any suspicion of plagiarism will be reported to the Dean of Faculty without delay. Both user and provider of such code will be reported and will be dealt with according to the plagiarism rules of the examination regulations (cf. OER, Article 118). The dean may decide on (a combination of) the disciplinary sanctions, ranging from 0/20 on the course to a prohibition from (re-)enrolling for one or multiple academic years (cf. OER, Article 118§5). The use of any third-party code, other than the libraries listed below, is forbidden. The use of generative AI is not allowed in any form. Contact us if you are in doubt as to whether or not something would be considered plagiarism.

Available Resources. In case you have any questions or need further clarification to complete the project, you can contact the assistant during the exercise sessions or by mail (noah.van.es@vub.be). A file `Main.hs` is provided on Canvas as a starting point for your implementation, containing both the typeclasses for which instances need to be provided (see below) and some (IO) code to run your application. Other than this file (and the `base` package), you are allowed to make use of the following third-party libraries:

- the `containers` package (<https://hackage.haskell.org/package/containers>)
- the `time` package (<https://hackage.haskell.org/package/time>)

You are allowed to use Haskell language extensions discussed during the exercise sessions.

Project Assignment

Your code should support the functionalities described below. **Ensure that your project can be compiled using version 9.6.7 of the GHC compiler².**

Hoodles. Irrespective of the representation, it should be possible to:

- Create a new hoodle with a given title.
- Display a hoodle as an ASCII-based table (as in the example given at the end of this document) containing its title, participants and timeslots. Timeslots should be displayed in ascending order of starting time.
- Add a timeslot to a hoodle with a given start and end time. Timeslots should be precise to the minute and work everywhere on earth. **It is not allowed for a hoodle to contain overlapping timeslots.**

²<https://www.haskell.org/ghc/>

- Remove a timeslot from a hoodle (based on its position).
- Register a participant to a hoodle.
- Unregister a participant to a hoodle. **This operation should also remove the user from every timeslot they voted on.**
- Add/remove (“toggle”) a name for a given timeslot (based on its position). **Only registered users are allowed to toggle a timeslot!**

To guide you in the right direction of a well-designed solution, you are required to implement the following Hoodle typeclass:

```
class Hoodle h where
  -- create an empty hoodle with a given title
  initialize :: String -> h t
  -- add a timeslot to the hoodle
  add :: Ord t => (t, t) -> h t -> Maybe (h t)
  -- remove a timeslot from the hoodle
  remove :: Int -> h t -> Maybe (h t)
  -- register a participant
  register :: String -> h t -> h t
  -- unregister a participant
  unregister :: String -> h t -> h t
  -- toggle a (registered) name to a timeslot
  toggle :: String -> Int -> h t -> Maybe (h t)
  -- get the title of a hoodle
  title :: h t -> String
  -- get all slots (including the users who voted for each slot)
  slots :: h t -> [(t, t, [String])]
  -- get all participants (registered users) of a hoodle
  participants :: h t -> [String]
```

You are required to provide two different implementations (i.e., two different instances of this typeclass): a “lazy” implementation, as well as an implementation where all hoodle operations are entirely strict (i.e., without any space leaks).

Pools. We will use a pool to store all hoodles, associating each hoodle with a key. It should therefore be possible to associate a new hoodle in the pool using a fresh (i.e., unused) key, allowing later access to the hoodle using this key. More specifically, you should implement the following Pool typeclass.

```
class Pool p where
  -- an empty pool
  emptyPool :: p k a
  -- adding an element to the pool
  addToPool :: (Enum k, Ord k) => a -> p k a -> (k, p k a)
  -- updating an existing element in the pool
  updatePool :: Ord k => k -> a -> p k a -> Maybe (p k a)
  -- getting an element from the pool
  getFromPool :: Ord k => k -> p k a -> Maybe a
  -- get all keys in use by the pool
  keys :: p k a -> [k]
```

Finding an Optimal Schedule. Given a collection of hoodles, you should be able to find a suitable schedule for all events/hoodles in that collection, so that:

- Each hoodle is assigned one of its possible slots.
- Each registered participant can attend the event; i.e., people registered to multiple events do not have to attend multiple events at the same time.

You are required to compute the optimal schedule. That is, the schedule which is backed by the greatest number of votes (equal to the sum of votes for each chosen slot).

Concretely, you need to implement a function `findOptimalSchedule`. **Determining the correct type for this function is part of the assignment.** Note that:

- The function should work for any hoodle representation (i.e., for any type `h` with a `Hoodle h` instance).
- The function should work for multiple collections of hoodles (e.g., a list of hoodles but also your own pool of hoodles).
- In some cases, it will not be possible to compute an optimal schedule, as no suitable schedule can be constructed without overlapping events for all participants.

Running the Application. For this programming assignment, a Haskell file `Main.hs` is available that contains IO code to interactively run the application, given your instances for the `Hoodle` and `Pool` typeclasses. Concretely, you can use the function `run` as follows:

```
main = run (emptyPool :: MyPool MyKey (MyHoodle MyTime))
```

Note that, in order to use function `run`, you have to **ensure that your choice of types satisfies all of the constraints in its type signature**.

Below, we illustrate its intended usage with an example.

```
Welcome to Hoodle!
Please make your selection:
[1] View hoodles
[2] Create a new hoodle
[3] Login as user
[4] Exit
>>> 2
Enter a title for your hoodle:
>>> Haskell Fan Club Party
+-----+
| Haskell Fan Club Party |
| (Participants: )      |
+-----+
Please make your selection:
[1] Add a timeslot
[2] Remove a timeslot
[3] Save hoodle to pool
[4] Cancel
>>> 1
Enter start time for slot:
>>> 2026-06-18 14:00:00 +0200
```

```

Enter end time for slot:
>>> 2026-06-18 16:00:00 +0200
Slot added.
+-----+
| Haskell Fan Club Party |
| (Participants: ) |
+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | |
+-----+
Please make your selection:
[1] Add a timeslot
[2] Remove a timeslot
[3] Save hoodle to pool
[4] Cancel
>>> 1
Enter start time for slot:
>>> 2026-06-19 10:00:00 +0200
Enter end time for slot:
>>> 2026-06-19 12:00:00 +0200
Slot added.
+-----+
| Haskell Fan Club Party |
| (Participants: ) |
+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | |
+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | |
+-----+
Please make your selection:
[1] Add a timeslot
[2] Remove a timeslot
[3] Save hoodle to pool
[4] Cancel
>>> 1
Enter start time for slot:
>>> 2026-06-18 20:00:00 +0200
Enter end time for slot:
>>> 2026-06-18 22:00:00 +0200
Slot added.
+-----+
| Haskell Fan Club Party |
| (Participants: ) |
+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | |
+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | |
+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | |
+-----+
Please make your selection:
[1] Add a timeslot
[2] Remove a timeslot
[3] Save hoodle to pool
[4] Cancel
>>> 3

```

```

Hoodle created! Hoodle ID: 0
Please make your selection:
[1] View hoodles
[2] Create a new hoodle
[3] Login as user
[4] Exit
>>> 3
Please enter your name:
>>> Alice
Currently logged in as: Alice
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 1
Enter the ID of the hoodle:
>>> 0
Done.
Currently logged in as: Alice
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 3
Enter the ID of the hoodle:
>>> 0
+-----+
| Haskell Fan Club Party |
| (Participants: Alice) |
+-----+-----+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | |
+-----+-----+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | |
+-----+-----+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | |
+-----+-----+-----+
Enter index of the slot to toggle:
>>> 0
Done.
Currently logged in as: Alice
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 3
Enter the ID of the hoodle:
>>> 0
+-----+
| Haskell Fan Club Party |
| (Participants: Alice) |
+-----+-----+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | Alice |

```

```

+-----+-----+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | |
+-----+-----+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | |
+-----+-----+-----+
Enter index of the slot to toggle:
>>> 1
Done.
Currently logged in as: Alice
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 4
Please make your selection:
[1] View hoodles
[2] Create a new hoodle
[3] Login as user
[4] Exit
>>> 3
Please enter your name:
>>> Bob
Currently logged in as: Bob
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 1
Enter the ID of the hoodle:
>>> 0
Done.
Currently logged in as: Bob
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 3
Enter the ID of the hoodle:
>>> 0
+-----+-----+-----+
| Haskell Fan Club Party |
| (Participants: Alice, Bob) |
+-----+-----+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | Alice |
+-----+-----+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | Alice |
+-----+-----+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | |
+-----+-----+-----+
Enter index of the slot to toggle:
>>> 2
Done.

```

```

Currently logged in as: Bob
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 3
Enter the ID of the hoodle:
>>> 0
+-----+
| Haskell Fan Club Party |
| (Participants: Alice, Bob) |
+-----+-----+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | Alice |
+-----+-----+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | Alice |
+-----+-----+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | Bob |
+-----+-----+-----+
Enter index of the slot to toggle:
>>> 1
Done.
Currently logged in as: Bob
Please make your selection:
[1] Register for a hoodle
[2] Unregister from a hoodle
[3] Vote for a hoodle slot
[4] Go back
>>> 4
Please make your selection:
[1] View hoodles
[2] Create a new hoodle
[3] Login as user
[4] Exit
>>> 1
Select the hoodle you want to view (or choose "Cancel" to go back)
[1] Haskell Fan Club Party (Hoodle ID: 0)
[2] Cancel
>>> 1
+-----+
| Haskell Fan Club Party |
| (Participants: Alice, Bob) |
+-----+-----+-----+
| 2026-06-18 14:00:00 +0200 | 2026-06-18 16:00:00 +0200 | Alice |
+-----+-----+-----+
| 2026-06-18 20:00:00 +0200 | 2026-06-18 22:00:00 +0200 | Alice, Bob |
+-----+-----+-----+
| 2026-06-19 10:00:00 +0200 | 2026-06-19 12:00:00 +0200 | Bob |
+-----+-----+-----+
Please make your selection:
[1] View hoodles
[2] Create a new hoodle
[3] Login as user
[4] Exit
>>> 4

```